

DATA SCIENCE AND MACHINE LEARNING

UNIT 04

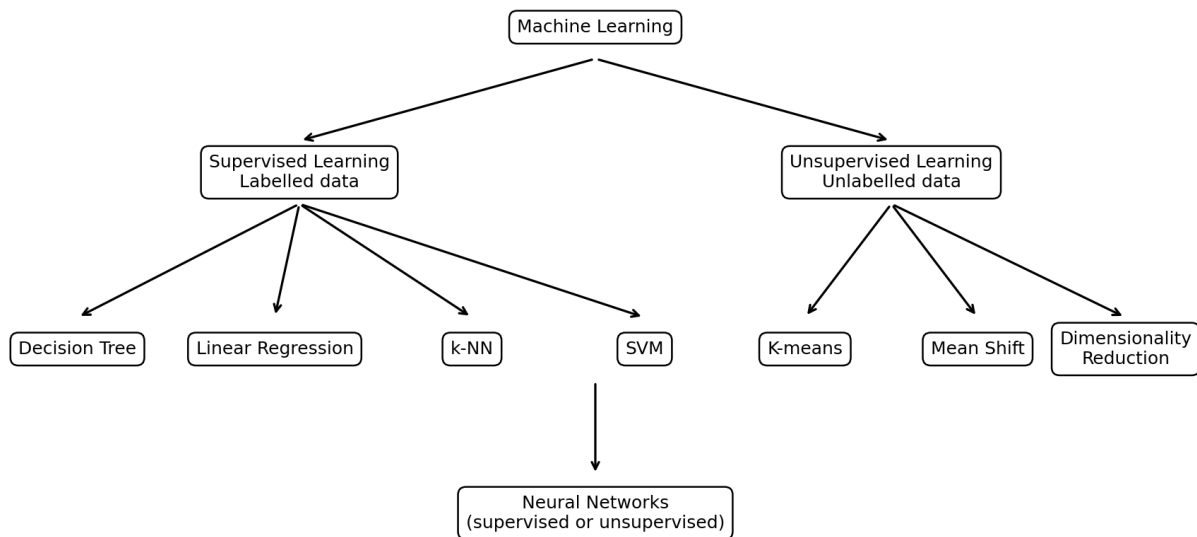
MACHINE LEARNING

ALGORITHMS

BEGINNER-FRIENDLY STUDENT NOTES

Syllabus coverage:

Unit 04 Algorithm Map



Unit 04 algorithm map: supervised algorithms learn from labelled data, while unsupervised algorithms discover structure in unlabelled data.

Learn the idea -> Follow the steps -> Read the code -> Interpret the result

A. How to Use These Notes

- Start with the problem type. First decide whether the required output is a class, a number, a group, or a smaller representation of the data.
- Understand the intuition before memorising formulas. Ask what information the algorithm uses and how it produces an answer.
- Work through each small numerical example by hand. Manual calculation makes the Python output meaningful.
- Run the Python examples and change one hyperparameter at a time. Observe how the prediction or grouping changes.
- Revise using the comparison tables, memory aids, formula sheet, quick checks and practice questions.

B. Learning Outcomes

- Differentiate supervised and unsupervised learning and identify classification, regression, clustering and dimensionality-reduction tasks.
- Explain the basic working of Decision Tree, Linear Regression, k-NN and SVM.
- Explain how K-means and Mean Shift create clusters from unlabelled data.
- Describe dimensionality reduction and the basic idea of Principal Component Analysis (PCA).
- Identify neurons, layers, weights, bias, activation functions, loss and backpropagation in a neural network.
- Select an introductory algorithm according to the target, data size, feature scale, interpretability and shape of the pattern.
- Implement small examples using NumPy, pandas and scikit-learn and interpret predictions and basic evaluation results.

C. Contents

1. Machine Learning Basics
2. A Common Workflow and Algorithm Selection
3. Decision Tree
4. Linear Regression
5. k-Nearest Neighbours (k-NN)
6. Support Vector Machine (SVM)
7. K-means Clustering
8. Mean Shift Algorithm
9. Dimensionality Reduction and PCA
10. Introduction to Neural Networks
11. Algorithm Comparison and Selection Guide
12. Integrated Python Practical
13. Formula Sheet and Glossary
14. Self-Assessment and Answer Key

1. Machine Learning Basics

Before studying individual algorithms, understand the words used in every machine learning problem.

1.1 What is a Machine Learning Algorithm?

A machine learning algorithm is a procedure that searches data for a useful pattern. During training, it creates or adjusts a model. The trained model then makes predictions or discovers structure in new data.

REMEMBER: Algorithm and model are not the same

The algorithm is the learning procedure. The model is the learned result. For example, Decision Tree is an algorithm; the particular tree created from a student dataset is the model.

1.2 Essential Vocabulary

Term	Simple meaning	Student example
Observation / sample	One row or one case in a dataset	One student
Feature (X)	Input information used for learning	Attendance, study hours, assignment score
Target (y)	Output that a supervised model must learn	Pass/Fail or final marks
Label	A categorical target value	Pass, Fail
Prediction	Output produced by the trained model	Predicted Pass
Parameter	Value learned from training data	Regression slope or network weight
Hyperparameter	Setting chosen before or during training	Tree depth, k in k-NN, number of clusters
Training	Learning patterns from examples	Fit model on training records
Inference / prediction	Using the trained model	Predict a new student result

1.3 Supervised and Unsupervised Learning

Point	Supervised learning	Unsupervised learning
Target available?	Yes. Every training example has a known output.	No target label is supplied.
Main question	Can we predict a known kind of output?	Can we discover hidden structure?
Common tasks	Classification and regression	Clustering and dimensionality reduction
Examples in this unit	Decision Tree, Linear Regression, k-NN, SVM	K-means, Mean Shift, PCA
Example	Predict Pass/Fail	Group students by learning behaviour

1.4 Four Problem Types

Problem type	Output	Example	Suitable introductory algorithms
Classification	A category or class	Spam / Not spam; Pass / Fail	Decision Tree, k-NN, SVM, Neural Network
Regression	A continuous number	House price; final marks	Linear Regression, Decision Tree Regressor, Neural Network
Clustering	A group discovered by the algorithm	Customer segments	K-means, Mean Shift

Problem type	Output	Example	Suitable introductory algorithms
Dimensionality reduction	Fewer features representing most information	Reduce 20 measurements to 2 components	PCA

QUICK CHECK

1. Predicting the exact temperature is classification or regression?
2. Grouping unlabelled shops by buying pattern is supervised or unsupervised?
3. What is the difference between a parameter and a hyperparameter?

2. A Common Workflow and Algorithm Selection

Different algorithms learn in different ways, but a reliable machine learning project follows a common sequence.

A Practical Machine Learning Workflow

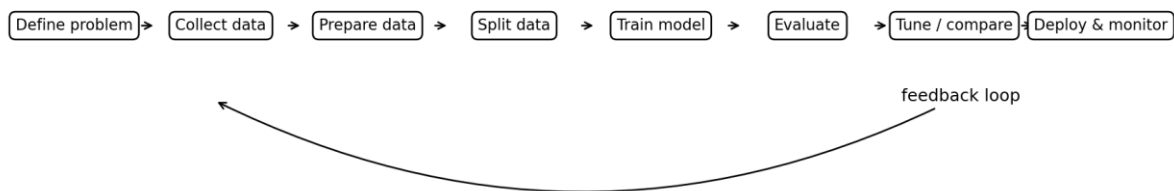


Figure 1: A basic machine learning workflow. Evaluation can send us back to data preparation or model selection.

2.1 Step-by-Step Workflow

1. Define the real problem and the expected output.
2. Collect representative data and understand each feature.
3. Clean missing values, duplicates and incorrect values.
4. Encode categorical features and scale features when required.
5. For supervised learning, split the data into training and test sets.
6. Fit the model only on the training data.
7. Predict test data and evaluate using suitable measures.
8. Tune hyperparameters, compare alternatives and check errors.
9. Explain limitations and monitor performance after deployment.

2.2 Training Data, Test Data and Generalisation

Training data teaches the model. Test data is kept separate and is used to estimate how well the learned pattern works on unseen cases. A model that performs well only on the training set has not generalised properly.

Condition	Meaning	Typical sign
Underfitting	Model is too simple to learn the pattern	Low training and low test performance
Good fit	Model learns useful pattern and generalises	Good and reasonably close training/test performance
Overfitting	Model memorises noise or details	Very high training but much lower test performance

CAUTION: Data leakage

Do not use information from the test set while fitting preprocessing steps or tuning the model. Leakage gives unrealistically high evaluation results.

2.3 Which Algorithms Need Feature Scaling?

Algorithm	Scaling importance	Why
Decision Tree	Usually not required	A tree compares one feature with a threshold.
Linear Regression	Helpful, especially for interpretation and numerical stability	Features may use very different units.
k-NN	Very important	Distances are dominated by large-scale features.
SVM	Very important	The margin and kernel depend on distances/dot products.
K-means	Very important	Clusters are created from distances to centroids.
Mean Shift	Important	Density neighbourhoods depend on feature distances.
PCA	Usually important	Large-variance units can dominate the components.
Neural Network	Very important	Optimisation is easier when inputs have comparable scales.

Safe split-and-scale pattern

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test) # transform only

```

3. Decision Tree

A Decision Tree predicts by asking a sequence of simple questions, similar to a flowchart.

A Small Decision Tree for Student Support

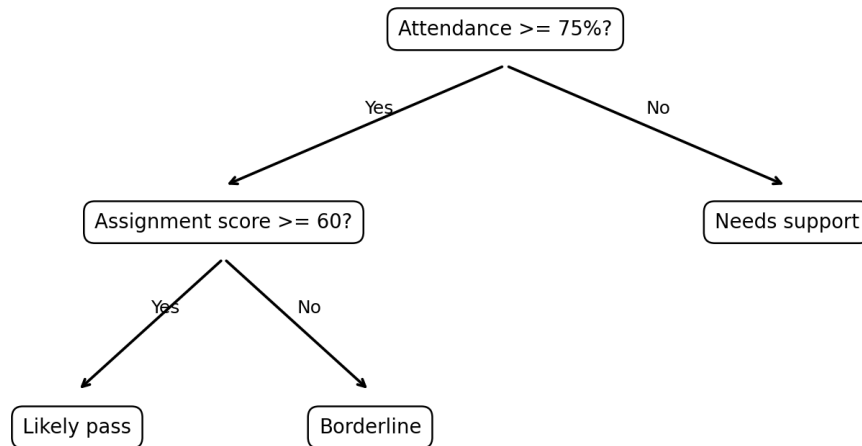


Figure 2: A decision tree contains a root, internal decision nodes, branches and leaf predictions.

3.1 Intuition

Suppose a teacher wants to predict whether a student will pass. The tree may first ask “Attendance >= 75%?” If yes, it may next ask “Assignment score >= 60?” Each answer follows a branch until a leaf gives the prediction.

Part	Meaning
Root node	First question asked by the tree
Internal node	A later question or test
Branch	Outcome of a question
Leaf node	Final prediction
Depth	Longest path from root to a leaf

3.2 How Does the Tree Select a Question?

The algorithm tests possible questions and chooses a split that makes the child groups purer. A pure group mostly contains one class. Gini impurity and entropy are common impurity measures.

Gini impurity

$$\text{Gini} = 1 - \sum(p_i^2)$$

p_i is the proportion of class i in the node. Gini is 0 for a completely pure node.

Example: A node has 6 Pass and 4 Fail. $\text{Gini} = 1 - (0.6^2 + 0.4^2) = 0.48$.

Entropy

$$\text{Entropy} = - \sum(p_i \log_2 p_i)$$

Entropy is also 0 for a pure node. Higher values mean more mixing.

Example: For 50% Pass and 50% Fail, entropy = 1 bit.

3.3 Small Split Example

Attendance	Assignment	Result
82	75	Pass
78	68	Pass
72	62	Pass
65	55	Fail
58	48	Fail
50	44	Fail

The question “Attendance ≥ 70 ?” separates all three Pass records from all three Fail records. Both resulting nodes are pure, so this is an excellent split for this small dataset. Real datasets are usually less perfect.

3.4 Overfitting and Tree Control

Hyperparameter	Effect
max_depth	Limits how many levels the tree may grow.
min_samples_split	Minimum samples required before a node can split.
min_samples_leaf	Minimum samples allowed in a leaf.
criterion	Impurity rule such as gini or entropy.
ccp_alpha	Controls cost-complexity pruning.

EXAM TIP: Why a deep tree can fail

A very deep tree can create leaves for tiny accidental patterns. It may perfectly classify training data but fail on new students. Control depth and compare test performance.

3.5 Python Example

Python example

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, pred))
```

Advantages	Limitations
Easy to explain as rules	Can overfit easily
Handles non-linear relationships	Small data changes can alter the tree

Advantages	Limitations
Little preprocessing required	Greedy splits are not always globally best
Works for classification and regression	Complex trees may be unstable

QUICK CHECK

1. What does a leaf contain?
2. Why is Gini 0 in a pure node?
3. Name two hyperparameters used to control overfitting.

4. Linear Regression

Linear Regression predicts a numerical value by fitting a straight line or a linear plane through the data.

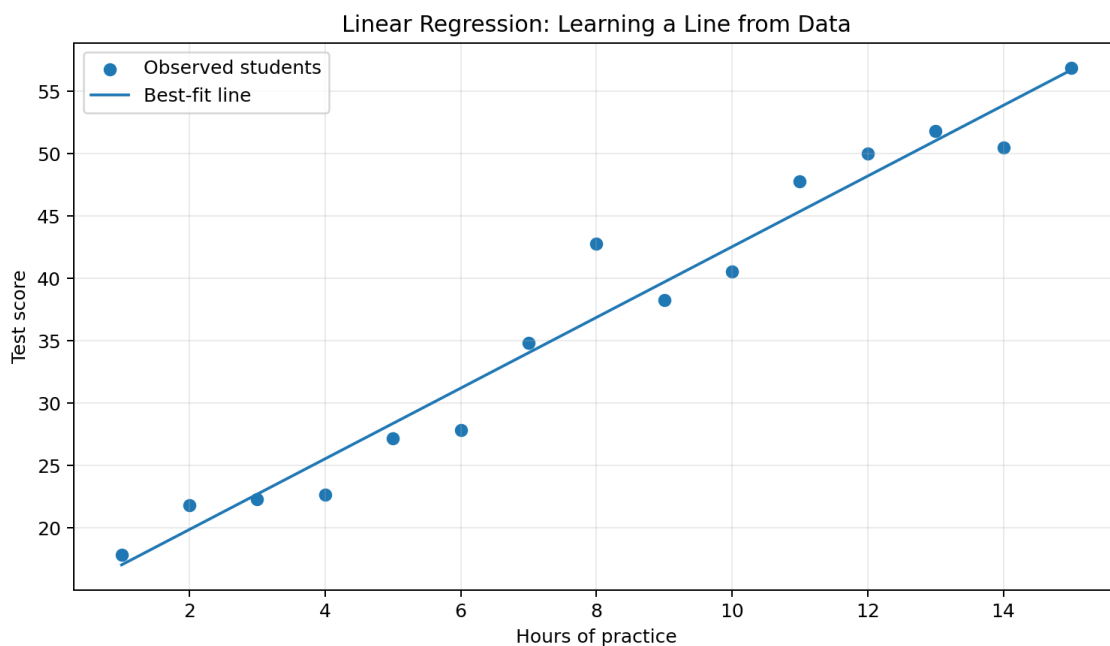


Figure 3: Linear regression chooses a line that keeps prediction errors as small as possible.

4.1 The Linear Equation

Simple linear regression

$$\hat{y} = b_0 + b_1 x$$

$\hat{y}$ is the predicted value, b_0 is the intercept, b_1 is the slope and x is the input feature.

Example: If predicted marks = $20 + 5 \times \text{study_hours}$, then 6 hours gives $20 + 5(6) = 50$ marks.

The slope tells how much the prediction changes when x increases by one unit. A positive slope means y tends to increase; a negative slope means y tends to decrease.

4.2 Residuals and Best-Fit Line

Residual

$$\text{residual} = \text{actual } y - \text{predicted } y$$

A residual is the vertical error made for one observation. Positive means the model predicted too low; negative means it predicted too high.

Example: Actual marks = 62, predicted marks = 58, residual = 4.

Mean Squared Error

$$\text{MSE} = (1/n) \sum (y_i - \hat{y}_i)^2$$

MSE averages squared prediction errors. Smaller values are better, but the unit is squared.

Example: Errors 2, -1, 3 give $\text{MSE} = (4 + 1 + 9) / 3 = 4.67$.

4.3 R-squared in Simple Language

R-squared (R^2) compares the regression model with a simple baseline that always predicts the mean target value. $R^2 = 1$ means perfect predictions on the evaluated data. $R^2 = 0$ means the model is no better than the mean baseline. A negative value means it is worse than that baseline.

IMPORTANT: Association is not causation

A useful regression relationship does not prove that changing x causes y to change. Hidden variables and study design still matter.

4.4 Assumptions at an Introductory Level

- The relationship is approximately linear.
- Observations are reasonably independent.
- Residual spread is roughly constant across prediction levels.
- Extreme outliers do not dominate the fitted line.
- For formal inference, residual distribution assumptions may also matter.

4.5 Python Example

Python example

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

X = np.array([[1], [2], [3], [4], [5], [6]])
y = np.array([24, 31, 36, 42, 47, 55])

model = LinearRegression()
model.fit(X, y)
pred = model.predict(X)

print('Intercept:', model.intercept_)
print('Slope:', model.coef_[0])
print('MAE:', mean_absolute_error(y, pred))
print('MSE:', mean_squared_error(y, pred))
print('R2:', r2_score(y, pred))
print('Prediction for 7 hours:', model.predict([[7]])[0])
```

Advantages	Limitations
Fast and easy to interpret	Cannot naturally learn strongly curved patterns
Good baseline for numerical prediction	Sensitive to influential outliers
Coefficients show direction and size	Correlation does not establish causation
Works with multiple features	Highly related features can complicate interpretation

5. k-Nearest Neighbours (k-NN)

k-NN predicts a new case by looking at the most similar stored training cases.

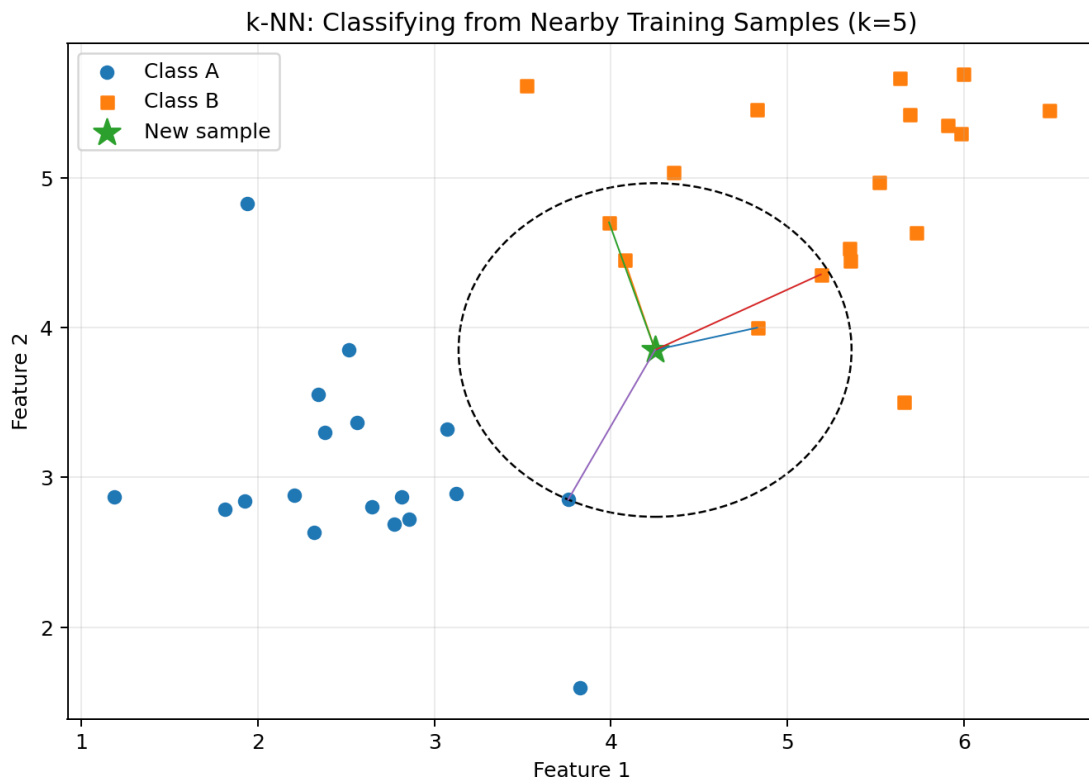


Figure 4: The new point receives a prediction from its nearest neighbours.

5.1 Intuition and Steps

1. Choose k , the number of neighbours.
2. Measure the distance from the new point to every training point.
3. Select the k smallest distances.
4. For classification, use majority vote. For regression, average neighbour targets.
5. Return the predicted class or value.

Euclidean distance in two dimensions

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Euclidean distance is the straight-line distance between two points.

Example: Distance from A(2,3) to B(5,7) = $\sqrt{3^2 + 4^2} = 5$.

5.2 Manual Classification Example

Point	Coordinates	Class	Distance from P(4,4)
A	(2, 3)	Blue	$\sqrt{5} = 2.24$
B	(3, 5)	Blue	$\sqrt{2} = 1.41$
C	(6, 5)	Red	$\sqrt{5} = 2.24$
D	(7, 7)	Red	$\sqrt{18} = 4.24$

For $k = 3$, the nearest points are B (Blue), A (Blue) and C (Red). Blue receives two votes, so P is classified as Blue.

5.3 Choosing k

Small k	Large k
Flexible and sensitive to local detail	Smoother decision boundary
Can be affected by noise	May ignore important local structure
Higher risk of overfitting	Higher risk of underfitting

CAUTION: Why scaling is essential

Suppose Age ranges from 18 to 60 but Annual Income ranges from 100000 to 2000000. Income differences will dominate Euclidean distance unless features are scaled.

5.4 Python Example with a Pipeline

Python example

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5)
)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, pred))
```

Advantages	Limitations
Simple and intuitive	Prediction becomes slow for large training sets
No complex training stage	Stores the training data
Can learn irregular boundaries	Sensitive to scaling and irrelevant features
Supports classification and regression	Performance drops in very high dimensions

6. Support Vector Machine (SVM)

SVM tries to separate classes with a boundary that leaves the widest possible margin.

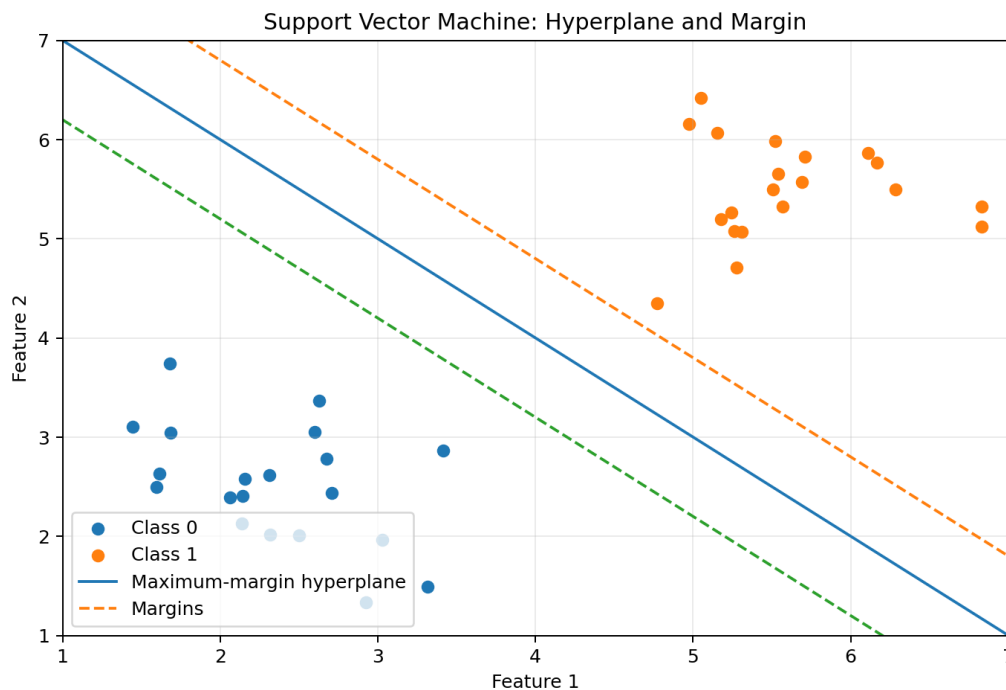


Figure 5: The support vectors are the closest points to the separating boundary and determine the maximum margin.

6.1 Hyperplane, Margin and Support Vectors

Term	Meaning
Hyperplane	The separating boundary. In 2D it is a line; in 3D it is a plane.
Margin	Distance between the boundary and the closest points from each class.
Support vectors	Training points closest to the boundary. They strongly determine its position.

Among many possible separating lines, SVM prefers the one with the widest margin because a wider margin often gives better generalisation.

6.2 Soft Margin and C

Real data may overlap, so SVM can allow some margin violations. The hyperparameter C controls the balance between a wide margin and training errors.

C value	Typical behaviour
Small C	Allows more violations and prefers a wider, smoother margin.
Large C	Punishes violations more strongly and tries harder to classify training points correctly.

6.3 Kernel Idea

If classes cannot be separated by a straight boundary, a kernel lets SVM represent a non-linear boundary. Common kernels are linear, polynomial and radial basis function (RBF). The RBF kernel is a common default for non-linear problems.

MEMORY AID: C and gamma are different

C controls the penalty for errors. With an RBF kernel, gamma controls how locally each training point influences the boundary. Very large values can produce an overly complex boundary.

6.4 Python Example

Python example

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

model = make_pipeline(
    StandardScaler(),
    SVC(kernel='rbf', C=1.0, gamma='scale')
)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, pred))
```

Advantages	Limitations
Effective in high-dimensional spaces	Feature scaling is important
Flexible non-linear kernels	Training can be slow on very large datasets
Uses important boundary points	Hyperparameters require careful tuning
Strong classifier for clear margins	Harder to explain than a small tree

QUICK CHECK

1. What are support vectors?
2. What is the purpose of the margin?
3. Why do SVM inputs normally require scaling?

7. K-means Clustering

K-means divides unlabelled observations into k groups by repeatedly updating cluster centres called centroids.

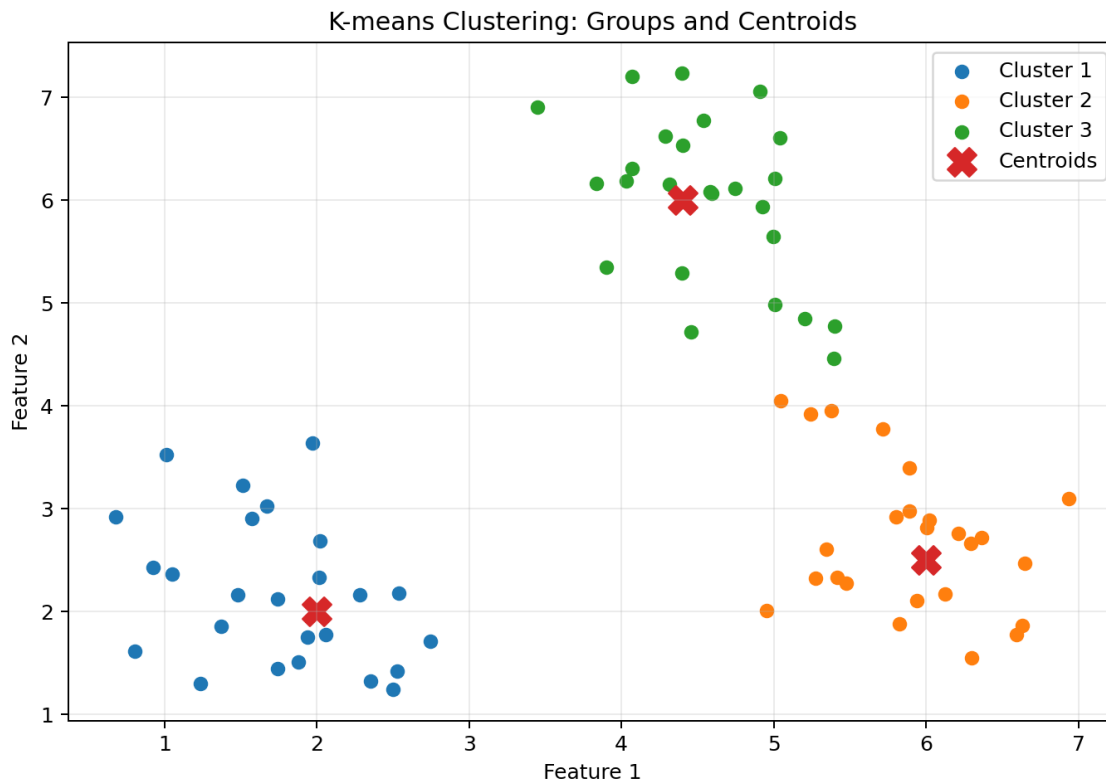


Figure 6: K-means alternates between assigning points to the nearest centroid and updating each centroid.

7.1 Algorithm Steps

1. Choose the number of clusters k .
2. Initialise k centroid positions.
3. Assign every point to its nearest centroid.
4. Replace each centroid with the mean of the points assigned to it.
5. Repeat assignment and update until the centroids stop changing significantly.

7.2 One-Dimensional Worked Example

Data points: 1, 2, 3, 10, 11, 12. Choose $k = 2$ and initial centroids $c1 = 2$ and $c2 = 10$.

Iteration	Cluster 1	Cluster 2	New centroids
1	1, 2, 3	10, 11, 12	$c1 = 2, c2 = 11$
2	1, 2, 3	10, 11, 12	No change; algorithm stops

The final clusters are the low-value group {1,2,3} and the high-value group {10,11,12}. The cluster numbers 0 and 1 are only identifiers; they do not carry a natural order.

7.3 Objective and Choosing k

Within-cluster squared error (inertia)

Inertia = sum of squared distances from each point to its assigned centroid

K-means tries to make clusters compact by reducing this value. Inertia always decreases when k increases, so it cannot select k alone.

- Elbow method: plot inertia for several k values and look for a point of diminishing improvement.

- Silhouette score: compares how close a point is to its own cluster versus other clusters.
- Domain knowledge: select a number of groups that can actually support a decision.

7.4 Important Limitations

- k must be selected in advance.
- Results can depend on initial centroids; multiple initialisations help.
- It works best for roughly compact, similar-sized clusters.
- Outliers can move the mean centroids.
- Feature scaling strongly affects distance and cluster formation.

7.5 Python Example

Python example

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

X, _ = load_iris(return_X_y=True)
X_scaled = StandardScaler().fit_transform(X)

model = KMeans(n_clusters=3, n_init=10, random_state=42)
labels = model.fit_predict(X_scaled)

print('Inertia:', model.inertia_)
print('Silhouette:', silhouette_score(X_scaled, labels))
print('First labels:', labels[:10])
```

8. Mean Shift Algorithm

Mean Shift discovers dense regions and does not require the number of clusters to be specified directly.

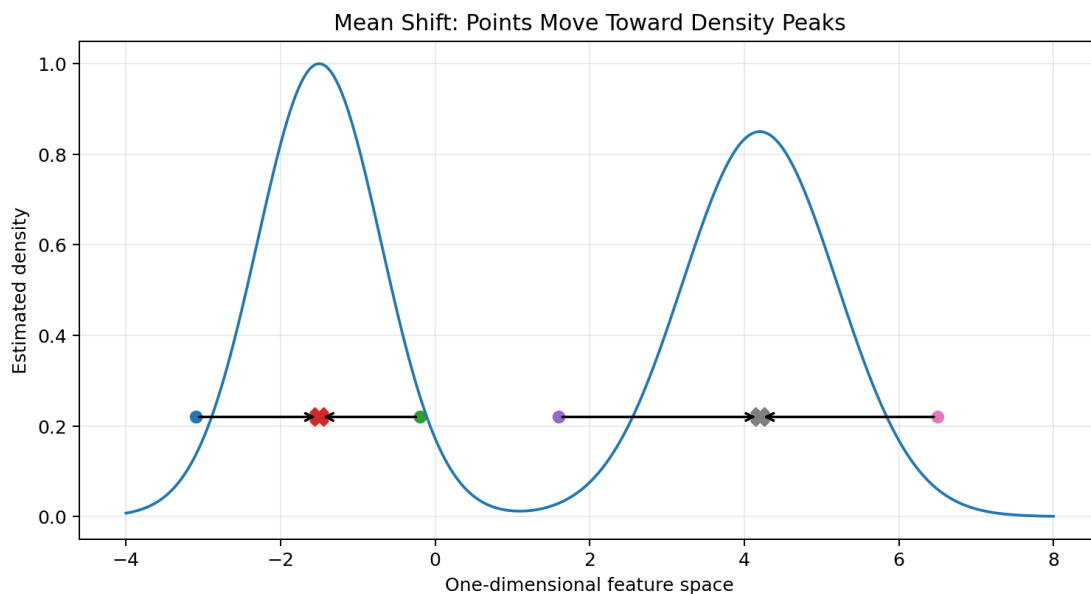


Figure 7: Mean Shift repeatedly moves a search window towards a nearby density peak.

8.1 Intuition

Imagine placing a circular window over a scatter plot. Calculate the mean position of the points inside the window and shift the window centre to that mean. Repeating this movement takes the centre towards a dense peak. Windows reaching the same peak form one cluster.

8.2 Bandwidth

Bandwidth controls the neighbourhood size. It is the most important Mean Shift setting.

Bandwidth	Typical effect
Too small	Many small clusters; sensitive to local noise
Reasonable	Density peaks represent meaningful groups
Too large	Different groups may merge into a few broad clusters

8.3 K-means versus Mean Shift

Point	K-means	Mean Shift
Number of clusters	k is supplied in advance	Emerges from density and bandwidth
Cluster centre	Arithmetic mean centroid	Density mode
Main control	k	Bandwidth
Speed	Usually faster	Often more computationally expensive
Best-known behaviour	Compact groups	Dense regions with unknown count

8.4 Python Example

Python example

```
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import MeanShift, estimate_bandwidth

X, _ = make_blobs(
    n_samples=200, centers=3, cluster_std=0.70, random_state=42
)
X_scaled = StandardScaler().fit_transform(X)

bandwidth = estimate_bandwidth(X_scaled, quantile=0.20)
model = MeanShift(bandwidth=bandwidth, bin_seeding=True)
labels = model.fit_predict(X_scaled)

print('Estimated clusters:', len(set(labels)))
print('Cluster centres:', model.cluster_centers_)
```

KEY IDEA: No target does not mean no evaluation

Clustering has no known y label during training, but we can still inspect plots, silhouette score, stability, cluster sizes and whether groups are useful for the real task.

9. Dimensionality Reduction and PCA

Dimensionality reduction represents data with fewer variables while trying to preserve useful information.

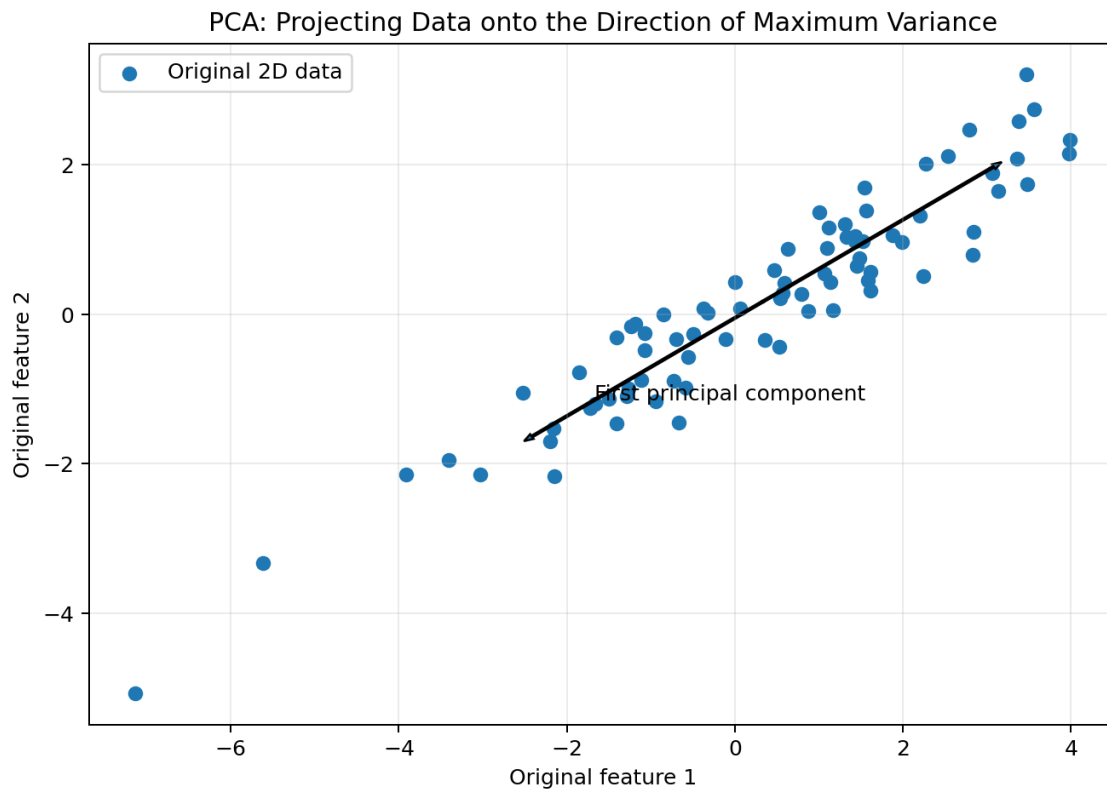


Figure 8: PCA rotates the coordinate system and keeps directions containing the greatest variation.

9.1 Why Reduce Dimensions?

- Visualise high-dimensional data in two or three dimensions.
- Reduce storage and computation.
- Remove redundancy from strongly correlated features.
- Reduce noise and sometimes improve downstream learning.
- Help distance-based methods when many irrelevant dimensions are present.

9.2 Feature Selection versus Feature Extraction

Method	What it does	Interpretation
Feature selection	Keeps a subset of original columns	Selected features retain their original names
Feature extraction	Creates new variables from the originals	New components combine several original features

PCA is a feature-extraction method. Principal components are new axes, not selected original columns.

9.3 Basic PCA Idea

1. Standardise features when their units or scales differ.
2. Centre the data around its mean.
3. Find the direction with maximum variance: Principal Component 1 (PC1).
4. Find a perpendicular direction with the next highest variance: PC2.
5. Keep enough components to preserve an acceptable amount of total variance.

REMEMBER: Components are ordered

PC1 explains the largest possible variance. PC2 explains the largest remaining variance while being perpendicular to PC1. Later components explain progressively less.

9.4 Explained Variance Ratio

The explained variance ratio reports the fraction of total variance captured by each component. For example, [0.72, 0.18] means PC1 explains 72% and PC2 explains 18%; together they preserve 90% of the variance.

9.5 Python Example

Python example

```
from sklearn.datasets import load_iris
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X, y = load_iris(return_X_y=True)

pca_model = make_pipeline(
    StandardScaler(),
    PCA(n_components=2)
)
X_2d = pca_model.fit_transform(X)

pca = pca_model.named_steps['pca']
print('New shape:', X_2d.shape)
print('Explained variance ratio:', pca.explained_variance_ratio_)
print('Total preserved:', pca.explained_variance_ratio_.sum())
```

Advantages	Limitations
Reduces feature count	Components may be difficult to interpret
Useful for visualisation	PCA is linear and may miss curved structure
Can remove correlated redundancy	High variance is not always the most predictive information
May speed later algorithms	Scaling and data leakage must be handled carefully

10. Introduction to Neural Networks

A neural network learns by combining many simple weighted calculations arranged in layers.

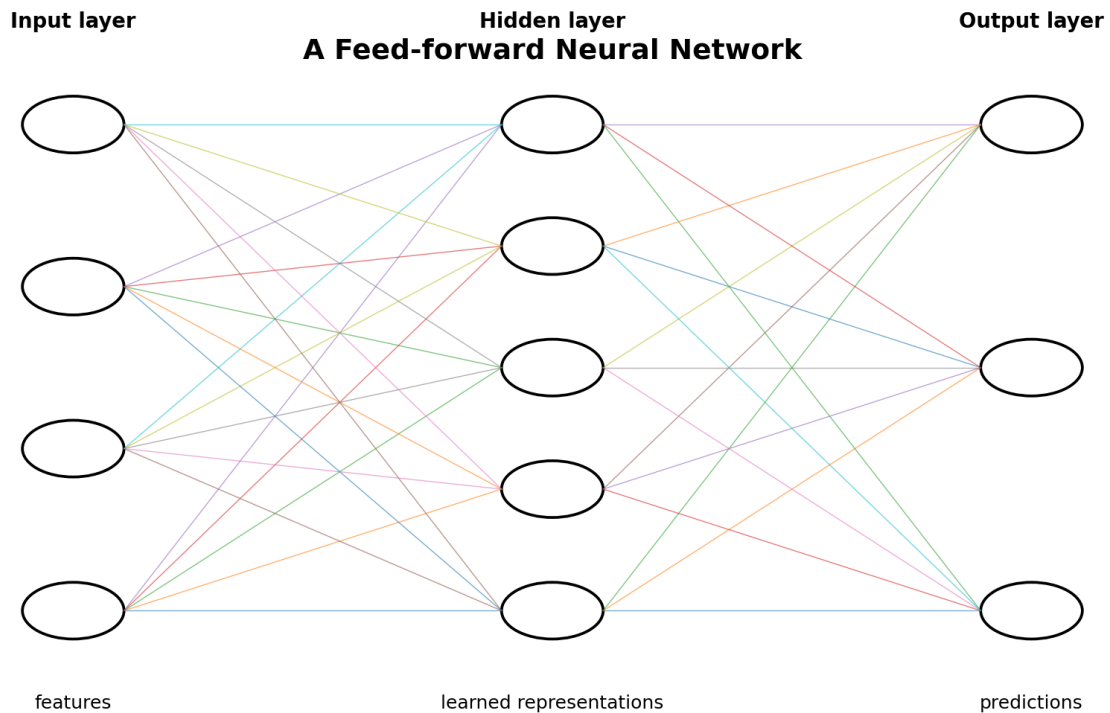


Figure 9: Inputs pass through hidden neurons to an output layer. Training adjusts weights and biases.

10.1 Artificial Neuron

Neuron calculation

$$z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b; \text{output} = \text{activation}(z)$$

Each input is multiplied by a weight. The bias shifts the result. An activation function adds non-linearity.

Example: With $x_1=2$, $x_2=3$, $w_1=0.5$, $w_2=-0.2$ and $b=0.4$, $z = 1 - 0.6 + 0.4 = 0.8$.

Part	Role
Input layer	Receives feature values
Hidden layer	Learns intermediate patterns
Output layer	Produces class scores or numerical predictions
Weight	Controls the strength and direction of an input connection
Bias	Allows a neuron to shift its activation threshold
Activation function	Introduces non-linearity
Loss function	Measures prediction error during training

10.2 Activation Functions

Activation	Basic idea	Common use
ReLU	$\max(0, z)$	Hidden layers
Sigmoid	Maps z between 0 and 1	Binary probability output
Tanh	Maps z between -1 and 1	Some hidden layers

Activation	Basic idea	Common use
Softmax	Converts scores into class probabilities summing to 1	Multiclass output

10.3 Training Cycle

1. Forward pass: calculate predictions using current weights.
2. Loss: compare predictions with known targets.
3. Backpropagation: calculate how each weight contributed to error.
4. Optimisation: update weights to reduce the loss.
5. Repeat for many epochs using batches of training examples.

Term	Meaning
Epoch	One complete pass through the training dataset
Batch	A subset used for one parameter update
Learning rate	Step size used when changing weights
Hidden units	Number of neurons in a hidden layer
Regularisation	Methods that reduce overfitting

10.4 Python Example: Multi-Layer Perceptron

Python example

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

model = make_pipeline(
    StandardScaler(),
    MLPClassifier(
        hidden_layer_sizes=(16,),
        max_iter=1000,
        random_state=42
    )
)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, pred))

```

KEY IDEA: Neural network is a family, not one fixed model

The number of layers, neurons, activations, loss functions and training settings define the architecture. A small MLP is suitable for introductory tabular examples; deep networks may use many specialised layers.

Advantages	Limitations
Learns complex non-linear relationships	Usually requires scaling and tuning
Flexible for classification and regression	Less interpretable than simple models

Advantages	Limitations
Can model interactions automatically	Can require more data and computation
Foundation of modern deep learning	May overfit without validation and regularisation

QUICK CHECK

1. What is the role of a weight?
2. Why is an activation function used?
3. What happens during backpropagation?

11. Algorithm Comparison and Selection Guide

There is no universally best algorithm. Select a reasonable starting model, evaluate it fairly and compare alternatives.

Algorithm	Task	Scaling	Interpretability	Important setting
Decision Tree	Classification / regression	Usually no	High for small trees	max_depth
Linear Regression	Regression	Helpful	High	Regularisation / features
k-NN	Classification / regression	Essential	Medium	n_neighbors
SVM	Classification / regression	Essential	Low-Medium	C, kernel, gamma
K-means	Clustering	Essential	Medium	n_clusters
Mean Shift	Clustering	Important	Medium	bandwidth
PCA	Dimensionality reduction	Usually essential	Low-Medium	n_components
Neural Network	Classification / regression	Essential	Low	layers, learning rate

11.1 Quick Selection Rules

Situation	Good first choice	Reason
Need simple explainable rules	Small Decision Tree	Predictions can be followed through questions
Predict a number with an approximately linear pattern	Linear Regression	Fast, transparent baseline
Local similarity is meaningful and data is not huge	k-NN	Uses nearby cases directly
Clear class boundary in scaled data	SVM	Max-margin classifier
Need k compact groups	K-means	Simple and efficient clustering
Number of dense groups is unknown	Mean Shift	Cluster count emerges from bandwidth
Too many correlated numeric features	PCA	Creates fewer components
Complex non-linear pattern with enough data	Neural Network	Flexible function approximator

11.2 Questions to Ask Before Choosing

- Is a target label available?
- Is the target categorical or numerical?
- How many observations and features are available?
- Are features on very different scales?
- Is an explainable model required?

- Is the expected pattern linear, local, tree-like, margin-based or highly non-linear?
- How much training time, prediction time and memory are available?
- Which evaluation measure reflects the real cost of mistakes?

PRACTICAL RULE: Start with a baseline

A complex algorithm is not automatically better. Begin with a simple baseline, use the same train/test split, preprocess correctly and compare models using suitable measures.

12. Integrated Python Practical

This practical compares supervised classifiers on the Iris dataset, then applies PCA and K-means to the same feature data.

12.1 Install and Import

Step 1: imports

```
# Run once in a terminal if the packages are not installed:
# pip install numpy pandas matplotlib scikit-learn

import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report
```

12.2 Load and Understand the Dataset

Step 2: dataset

```
iris = load_iris(as_frame=True)
df = iris.frame

print(df.head())
print(df.shape)
print(df['target'].value_counts())
print(iris.target_names)

X = df.drop(columns='target')
y = df['target']
```

12.3 Create One Fair Train-Test Split

Step 3: split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

12.4 Compare Three Classifiers

Step 4: model comparison

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

models = {
    'Decision Tree': DecisionTreeClassifier(
        max_depth=3, random_state=42
    ),
    'k-NN': make_pipeline(
        StandardScaler(), KNeighborsClassifier(n_neighbors=5)
    ),
    'SVM': make_pipeline(
        StandardScaler(), SVC(kernel='rbf', C=1.0)
    )
}

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(name, 'accuracy =', accuracy_score(y_test, pred))
```

INTERPRETATION: Fair comparison

All models use the same training and test observations. Scaling is placed inside pipelines for k-NN and SVM so it is fitted only on training data.

12.5 Apply PCA and K-means

Step 5: representation and clustering

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

X_scaled = StandardScaler().fit_transform(X)
X_2d = PCA(n_components=2).fit_transform(X_scaled)

cluster_model = KMeans(
    n_clusters=3, n_init=10, random_state=42
)
clusters = cluster_model.fit_predict(X_2d)

print('Reduced shape:', X_2d.shape)
print('Silhouette:', silhouette_score(X_2d, clusters))
print(pd.crosstab(clusters, y))
```

The crosstab compares discovered cluster IDs with known species labels only for interpretation. K-means did not use y while creating clusters, and cluster numbers do not have to match class numbers.

12.6 Optional Visualisation

Step 6: plot clusters

```
import matplotlib.pyplot as plt

plt.scatter(X_2d[:, 0], X_2d[:, 1], c=clusters)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('K-means Clusters after PCA')
plt.show()
```

13. Formula Sheet and Glossary

13.1 Formula Sheet

Concept	Formula / rule	Use
Gini impurity	$1 - \sum(p_i^2)$	Measures class mixing in a tree node
Entropy	$-\sum(p_i \log_2 p_i)$	Alternative tree impurity measure
Linear regression	$\hat{y} = b_0 + b_1 x$	Predicts a number from a line
Residual	$y - \hat{y}$	Error for one prediction
MSE	$(1/n) \sum((y - \hat{y})^2)$	Average squared regression error
Euclidean distance	$\sqrt{\sum((a_i - b_i)^2)}$	Distance in k-NN and clustering
Neuron input	$z = \sum(w_i x_i) + b$	Weighted combination before activation
ReLU	$\max(0, z)$	Common hidden-layer activation

13.2 Glossary

Term	Meaning
Activation function	Function that adds non-linearity to a neural network neuron.
Bandwidth	Neighbourhood size controlling Mean Shift density estimation.
Centroid	Mean position representing a K-means cluster.
Classification	Supervised prediction of a category.
Cluster	Group of observations with similar feature patterns.
Component	New direction created by a dimensionality-reduction method such as PCA.
Decision boundary	Region or line separating predicted classes.
Epoch	One full pass through neural-network training data.
Feature	Input variable used by a model.
Generalisation	Ability to perform well on unseen data.
Hyperparameter	User-controlled setting such as k, C or max_depth.
Inertia	K-means sum of squared distances to assigned centroids.
Kernel	SVM method for representing non-linear decision boundaries.
Label	Known categorical target value.
Margin	Gap between an SVM boundary and closest class points.
Model	Pattern learned by an algorithm from data.
Overfitting	Learning training details that do not generalise.
Parameter	Value learned during training.
Regression	Supervised prediction of a continuous number.
Support vector	Training point that helps determine the SVM boundary.
Target	Output variable a supervised model learns to predict.
Unsupervised learning	Learning structure without known target labels.
Weight	Learned strength of a neural-network connection.

14. Self-Assessment and Answer Key

14.1 Multiple-Choice Questions

1. Which task predicts a continuous value?
A. Classification B. Regression C. Clustering D. PCA
2. Which algorithm usually does not require feature scaling?
A. k-NN B. SVM C. Decision Tree D. K-means
3. A final prediction node in a decision tree is called a:
A. Root B. Leaf C. Margin D. Centroid
4. Gini impurity of a pure node is:
A. 0 B. 0.5 C. 1 D. Undefined
5. In $y_{\text{hat}} = b_0 + b_1x$, b_1 is the:
A. Target B. Slope C. Residual D. Cluster
6. k-NN classification normally uses:
A. Mean target B. Majority vote C. Maximum margin D. Principal component
7. The closest SVM points to the boundary are:
A. Centroids B. Support vectors C. Outliers D. Components
8. Which SVM setting controls error penalty?
A. k B. C C. Bandwidth D. n_components
9. K-means requires the user to provide:
A. Class labels B. Number of clusters k C. Regression slope D. Support vectors
10. The representative centre of a K-means group is a:
A. Centroid B. Kernel C. Leaf D. Neuron
11. Mean Shift cluster count mainly depends on:
A. Tree depth B. Bandwidth C. R² D. Epoch
12. PCA creates:
A. Only class labels B. New principal components C. Tree branches D. Cluster targets
13. PC1 captures:
A. Minimum possible variance B. Maximum possible variance C. Only target variance D. No variance
14. A neural-network bias is:
A. A data error B. An adjustable offset C. A cluster number D. A test set
15. Backpropagation is used to:
A. Choose k B. Update weights using error information C. Create centroids D. Sort neighbours
16. An epoch means:
A. One feature B. One full pass through training data C. One test record D. One hidden neuron
17. Very high training and low test performance suggests:
A. Underfitting B. Overfitting C. Perfect generalisation D. Clustering
18. Which method is unsupervised?
A. Linear Regression B. SVC C. K-means D. Decision Tree Classifier
19. Which algorithms are strongly distance-based?
A. k-NN and K-means B. Tree and Linear Regression only C. Decision Tree and PCA only D. None
20. A simple model used for comparison is called a:
A. Baseline B. Kernel C. Support vector D. Mode

14.2 Short-Answer Questions

1. Differentiate an algorithm and a model.

2. Differentiate classification and regression with one example each.
3. Why is a separate test set required?
4. Explain root, branch and leaf in a decision tree.
5. What is a residual in linear regression?
6. Why is feature scaling important for k-NN?
7. What are support vectors?
8. Write the five main steps of K-means.
9. What is bandwidth in Mean Shift?
10. Differentiate feature selection and feature extraction.
11. What does explained variance mean in PCA?
12. Name the main parts of an artificial neuron.

14.3 Long-Answer Questions

1. Explain Decision Tree classification with impurity, split selection, overfitting controls, advantages and limitations.
2. Explain Linear Regression using the line equation, residuals, MSE, R², assumptions and a suitable example.
3. Describe k-NN classification with a manual distance example. Discuss k selection, scaling, strengths and limitations.
4. Explain SVM using hyperplane, margin, support vectors, C and kernels.
5. Compare K-means and Mean Shift clustering.
6. Explain PCA, its steps, explained variance, uses and limitations.
7. Draw and explain a feed-forward neural network. Describe forward pass, loss, backpropagation and weight update.
8. Compare all Unit 04 algorithms according to task, scaling, interpretability and important hyperparameters.

14.4 Practical Exercises

1. Train DecisionTreeClassifier on the Iris dataset. Change max_depth from 1 to 5 and compare training and test accuracy.
2. Create a small study-hours dataset and use LinearRegression to predict marks for a new value.
3. Train k-NN for k = 1, 3, 5, 7 and 9. Record test accuracy and identify the best k for the chosen split.
4. Compare linear and RBF SVM pipelines on a classification dataset.
5. Apply K-means for k = 2 to 6 and record inertia and silhouette score.
6. Apply PCA to four Iris features and preserve two components. Plot the transformed data.
7. Train MLPClassifier with one hidden layer and compare its result with a Decision Tree.

14.5 MCQ Answer Key

1-5	6-10	11-15	16-20
1-B, 2-C, 3-B, 4-A, 5-B	6-B, 7-B, 8-B, 9-B, 10-A	11-B, 12-B, 13-B, 14-B, 15-B	16-B, 17-B, 18-C, 19-A, 20-A

14.6 One-Minute Revision

- Tree asks questions; Linear Regression fits a line.
- k-NN follows nearby examples; SVM maximises margin.
- K-means needs k; Mean Shift needs bandwidth.
- PCA creates fewer components; Neural Networks combine weighted neurons.
- Scale distance-, margin-, component- and gradient-based methods.
- Evaluate on unseen data and compare against a baseline.

Further Reading

The Python examples use the scikit-learn estimator and pipeline style. Official reference: [scikit-learn User Guide](#)

Documentation interface checked in July 2026. The concepts remain applicable across machine learning software, but exact defaults may vary by version.